

M0487 – Entorns de desenvolupament



Introducció

□ Cada vegada ens trobem més dispositius a la vida quotidiana que executen algun tipus de software.

- Ordinadors
- Tauletes
- Telèfons mòbils
- Neveres
- Llibres electrònics
- Caixers automàtics
- Impressores i fotocopiadores
- Videoconsoles
- Televisors
- Automoció
- etc.



Introducció

- ❑ Tots aquests dispositius cal programar-los i, per tant, tot el software que hi corre s'ha hagut de definir, dissenyar, desenvolupar i mantenir.
- ❑ El desenvolupament de programari organitzat consta d'una sèrie de fases.
- ❑ En aquesta UF veurem aquestes diferents fases a seguir, els rols que hi intervenen i algunes eines que ens poden ajudar a desenvolupar aquestes tasques.

El programa informàtic

1.1. El programa Informàtic

❑ Sense informar-se a internet... Responen la següent pregunta...

❑ Què és un programa informàtic?



1.1. El programa Informàtic

- ❑ Un **programa informàtic** és un **conjunt d'instruccions** que s'executen de manera **seqüencial** amb l'objectiu de realitzar una o diverses tasques en un sistema.
- ❑ És creat per un programador o **equip de programació** també anomenat **equip de desenvolupament**.
- ❑ Utilitzant un o diversos **llenguatges** determinats.
- ❑ Les instruccions que generi l'equip de desenvolupament, s'anomenen **codi font**. Aquest, serà **compilat i executat** per un sistema.



1.1.1. Interacció amb el sistema

Software Actual

- ❑ Les màquines reals i actuals tenen un funcionament intern similar.
- ❑ Hi ha software que té accés directe al hardware i n'hi ha que no. Classificació:

- ❑ **Software de sistema:**

- Gestiona el hardware.
- Interacciona amb l'usuari directament o amb el software d'aplicació.
- Formen els sistemes operatius: Linux, Mac OS, Android, Windows, etc.



- ❑ **Software d'aplicació:**

- Interactua amb l'usuari final i amb el software de sistema.
- Té una utilitat concreta i específica.
- Adjunta les llibreries necessàries pel SO.



- ❑ Ambdós tipus de software cal desenvolupar-los a partir d'un llenguatge de programació.



Classificacions dels llenguatges de programació

1.2. Llenguatges de programació

- ❑ Un **llenguatge de programació** és un **conjunt d'instruccions, operadors i regles sintàctiques i semàntiques** que podem utilitzar per gestionar els recursos HW i SW existents al sistema.
- ❑ El processador només entén **codi màquina** (0s i 1s).
- ❑ Els LP permeten escriure codi de forma més semblant al **llenguatge natural**, tot i que sempre acabarà sent traduït internament a codi màquina pel **compilador**.
- ❑ El primer LP va ser **FORTRAN** (FORmula TRANslation).

1.2.1. Classificació dels LP

□ Podem classificar els LP per:

- Nivell d'abstracció
- Forma d'execució
- Paradigma de la programació

1.2.1. Classificació dels LP

Nivell d'abstracció

❑ El **Nivell d'abstracció** d'un LP és si les seves instruccions s'acosten més al **codi màquina** (**baix nivell**) o al **llenguatge natural** (**alt nivell**).

❑ Els diferenciarem en tres grans tipus, llenguatges de:

- Baix nivell
- Mitjà nivell
- Alt nivell

1.2.1. Classificació dels LP

Nivell d'abstracció

- *Llenguatges de baix nivell*
 - Primera generació:
 - Codi màquina (0s i 1s)
 - La màquina ho executa directament sense necessitat d'interpretació



1.2.1. Classificació dels LP

Nivell d'abstracció

- *Llenguatges de mitjà nivell*
 - Segona generació:
 - Neix per fer una mica més amena la programació en codi màquina
 - Poques instruccions i per fer operacions senzilles
 - Treballa directament sobre els registres i les posicions de memòria de la màquina
 - El llenguatge clau de la segona generació és el **llenguatge ensamblador**



1.2.1. Classificació dels LP

Nivell d'abstracció

- ***Llenguatges d'alt nivell***

- La majoria de LP actuals
- Programació més intel·ligible i intuïtiva
- Molt semblant al llenguatge natural
- Acostumen a tenir unes llibreries que ajuden al programador.
- Permeten la Programació orientada a objectes.
- Ofereixen *Frameworks* per una programació més eficient i ràpida.
- Comuns als àmbits de la intel·ligència artificial i la lògica.

```
public static string DigitsOnlyTest1(string strRawData)
{ // Using Dot Net 2.0 - - Extract the digits out of a string.
  StringBuilder sbRetVal = new StringBuilder();

  foreach (char chr in strRawData)
  {
    if ((chr < '0') || (chr > '9'))
    {
      continue;
    }
  }
}
```

1.2.1. Classificació dels LP

Nivell d'abstracció

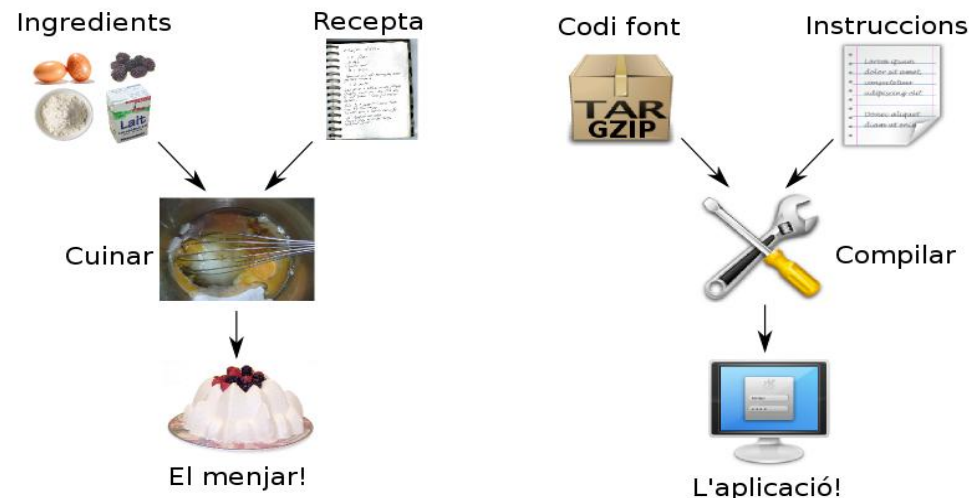
- Un ***framework*** es un conjunt de conceptes, estructures, funcions, components, etc. que ajuden al programador a accedir a eines per personalitzar els llenguatges i adaptar-ho a les seves necessitats.
- *Exemples: Bootstrap (CSS + Javascript), Laravel (PHP), Symfony (PHP), Angular (Javascript), ReactJS (Javascript), Vue.js (Javascript), Hibernate (Java), ASP.Net (C#)...*

1.2.1. Classificació dels LP

Forma d'execució

- *Llenguatges compilats*

- El **compilador** tradueix el codi font a codi objecte
- L'**enllaçador** uneix el codi objecte del programa amb el codi objecte de les llibreries necessàries per a fer l'executable.



L'executable funciona per a un sistema operatiu determinat.

1.2.1. Classificació dels LP

Forma d'execució

- *Llenguatges interpretats*
 - L'interpret va executant les instruccions directament una a una seqüencialment
 - Traducció a temps real només de les instruccions a executar
 - L'interpret ha d'estar instal·lat al sistema
 - Execució més lenta

1.2.1. Classificació dels LP

Forma d'execució

- *Llenguatges virtuals*
 - Similar als llenguatges compilats
 - No genera codi objecte (per a un tipus de HW específic)
 - Genera un **bytecode** que es podrà executar en qualsevol màquina virtual que contingui l'interpret adequat
 - És **multisistema** i **multiplataforma**
 - No depèn del hardware

1.2.1. Classificació dels LP

Paradigma de la programació

- El **Paradigma de programació** és l'enfocament que se li dona a la programació, la manera, l'estil per crear el programa.
- Afegeix funcionalitats i facilita la programació.
- Tots pertanyen a llenguatges d'alt nivell.
- Un LP pot utilitzar diferents paradigmes.
- Triarem un tipus o un altre depenent del propòsit del programa.

1.2.1. Classificació dels LP

Paradigma de la programació

- *Paradigma imperatiu*

- Seqüència d'instruccions que van definint **COM** actua el programa en cada situació.
- Èmfasi en els canvis d'estat mitjançant la mutació de variables.

- *Paradigma declaratiu*

- Es declaren les premisses i condicions que indiquen **QUÈ** s'ha de fer però no necessàriament el COM.

- *Paradigma procedimental*

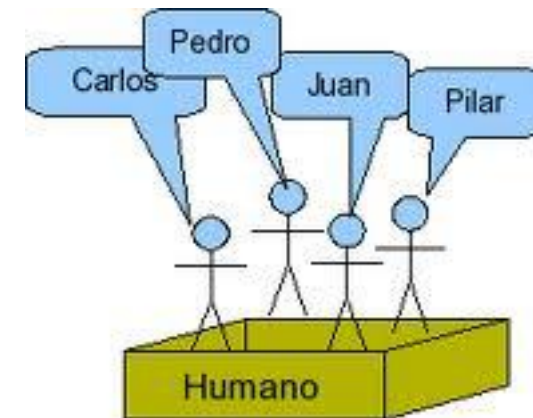
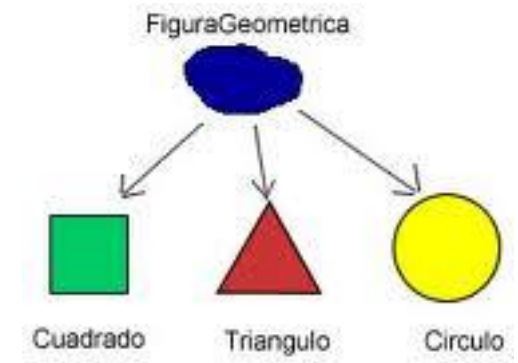
- Es divideix el programa en blocs més petits, anomenats **funcions** i **procediments**, que fan tasques específiques i que es comuniquen entre ells.
- Permet reutilitzar codi.

1.2.1. Classificació dels LP

Paradigma de la programació

- *Paradigma orientat a objectes*

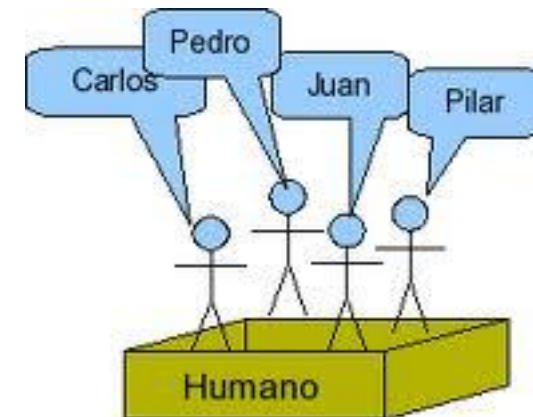
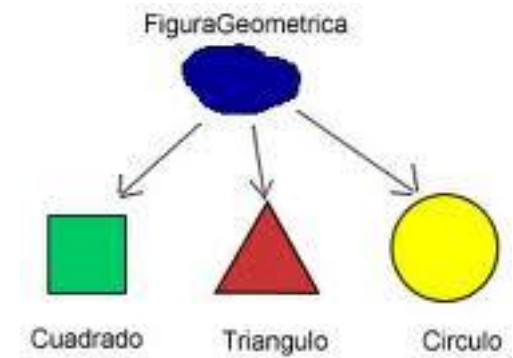
- Es defineixen **objectes** amb determinades **propietats** i **operacions** que poden realitzar o que podem realitzar amb ells.
- Es defineix una estructura de **classes** i **objectes** (instàncies) que emulen el món real.
- Cada objecte pot realitzar determinades accions i interactua amb altres objectes.
- Es poden definir relacions d'herència entre classes i objectes, creant així una abstracció conceptual similar al món real.
- Permet reutilitzar molt de codi.
- Simplifica enormement la tasca de programació, de depuració i manteniment del programari.
- Redueix la possibilitat d'errors.



1.2.1. Classificació dels LP

Paradigma de la programació

- *Paradigma orientat a objectes*
 - Basat en les següents tècniques:
 - Jerarquia (herència)
 - Abstracció
 - Polimorfisme
 - Encapsulació i principi d'ocultació
 - Modularitat



1.2.1. Classificació dels LP

Paradigma de la programació

- **Paradigma funcional**

- Es basa en la definició recursiva de **funcions matemàtiques**.
- Evita al màxim declarar dades.
- Es centra en la composició de les funcions i les interaccions entre elles.



- **Paradigma lògic**

- Es defineix un conjunt de **regles lògiques (premisses)** que seran interpretades mitjançant **inferències lògiques (motor)**.
- És idoni per la **resolució de problemes**.

$R \vee \text{no } Q$	
$P \Rightarrow Q$	Premisas
$\text{no}(R \vee P)$	
$\text{no } P$	Conclusión
1. $R \vee \text{no } Q$	Premisa
2. $P \Rightarrow Q$	Premisa
3. $\text{no}(R \vee P)$	Premisa
4. Supongamos: P	Hipótesis auxiliar. Reducción al absurdo.
5. Q	Modus Ponens de 4. y 2.
6. R	Modus Tollendo Ponens de 5. y 1.
7. $R \vee P$	Conjunción 4. y 6.
8. $(R \vee P) \vee \text{no}(R \vee P)$	Conjunción 3. y 7. (Contradicción).
9. $\text{no } P$	Reducción al absurdo.

Obtenció de codi executable

1.3. Obtenció de codi executable

- ❑ Un programa el desenvolupem en un llenguatge i sobre un entorn determinats, però per poder ser executat, finalment ha de ser traduït, d'una manera o altra, a codi màquina.

1.3. Obtenció de codi executable

1.3.1. Tipus de codi

❑ Les etapes de codificació per les que passarà el nostre programa són les següents:

- Codi font
- Codi objecte
- Codi executable

1.3. Obtenció de codi executable

1.3.1. Tipus de codi

- *Codi font*

- És el codi que genera el programador.
- Estarà en el **llenguatge escollit** pel desenvolupament.

- *Codi objecte*

- És el resultat de compilar el codi font.
- Si és un llenguatge compilat, el resultat és **codi màquina**.
- Si és un llenguatge virtual, el resultat és **bytecode**.

1.3. Obtenció de codi executable

1.3.1. Tipus de codi

- ***Codi executable***

- És el resultat d'enllaçar el codi objecte amb les **llibries** de sistema requerides.
- Ja és codi que **es pot executar** directament sobre el sistema o sobre una màquina virtual.

En el cas de **llenguatges interpretats** tenim una **excepció**, ja que no es genera ni codi objecte ni executable • el codi font és **compilat i executat** línia a línia per un intèrpret **en temps real**.

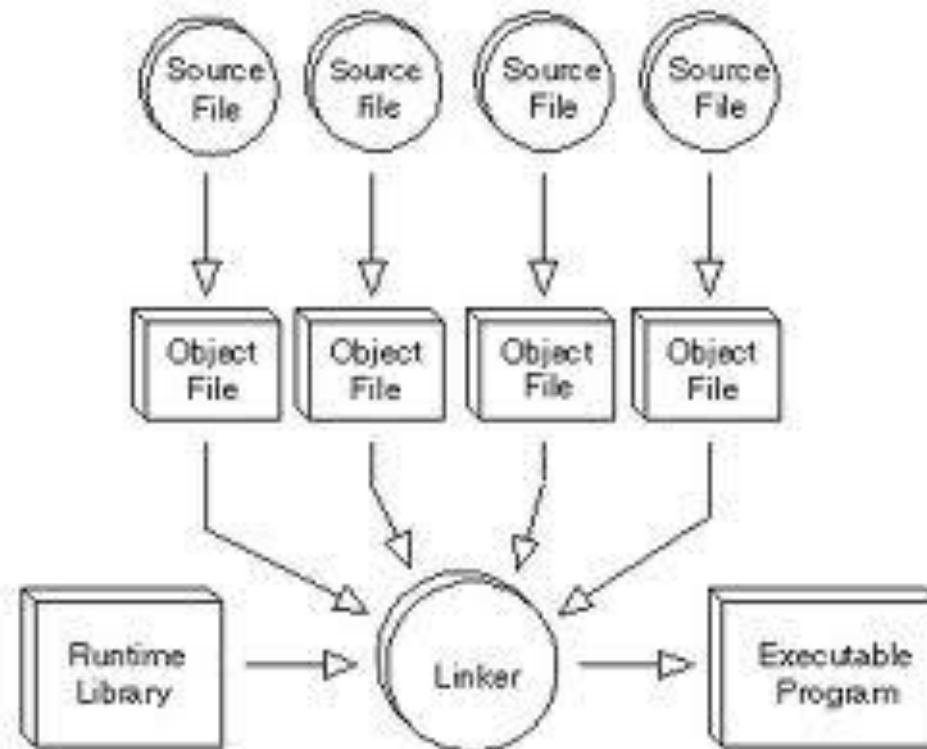
1.3. Obtenció de codi executable

1.3.2. Compilació

□ “**Compilar**” = **compilar** + **enllaçar**

(codi objecte)

(llibreries)



1.3. Obtenció de codi executable

1.3.2. Compilació – Fases

1. Anàlisi lexicogràfic

- Es llegeix tot el codi font de manera seqüencial.
- Es busquen les **paraules reservades**, **operacions**, **caràcters de puntuació**.
- Es fa el reconeixement de **lexemes**.

2. Anàlisi sintàctic-semàntic

- Agrupa els lexemes en frases gramaticals.
- Es revisa la coherència de les frases gramaticals:
 - Si la sintaxi i la semàntica de les instruccions és correcta.
 - Si els tipus de dades definits quadra amb el seu ús.
 - Si falta algun signe de puntuació.

1.3. Obtenció de codi executable

1.3.2. Compilació – Fases

3. Generació de codi intermig

- A partir del resultat de l'anàlisi es genera un codi intermig en **pseudoassemblador** que facilitarà la generació del codi objecte.

4. Optimització de codi

- Es revisa el codi pseudoassemblador generat i s'optimitza de cara a que el codi resultant sigui **més fàcil i ràpid d'executar** per la màquina.

5. Generació de codi

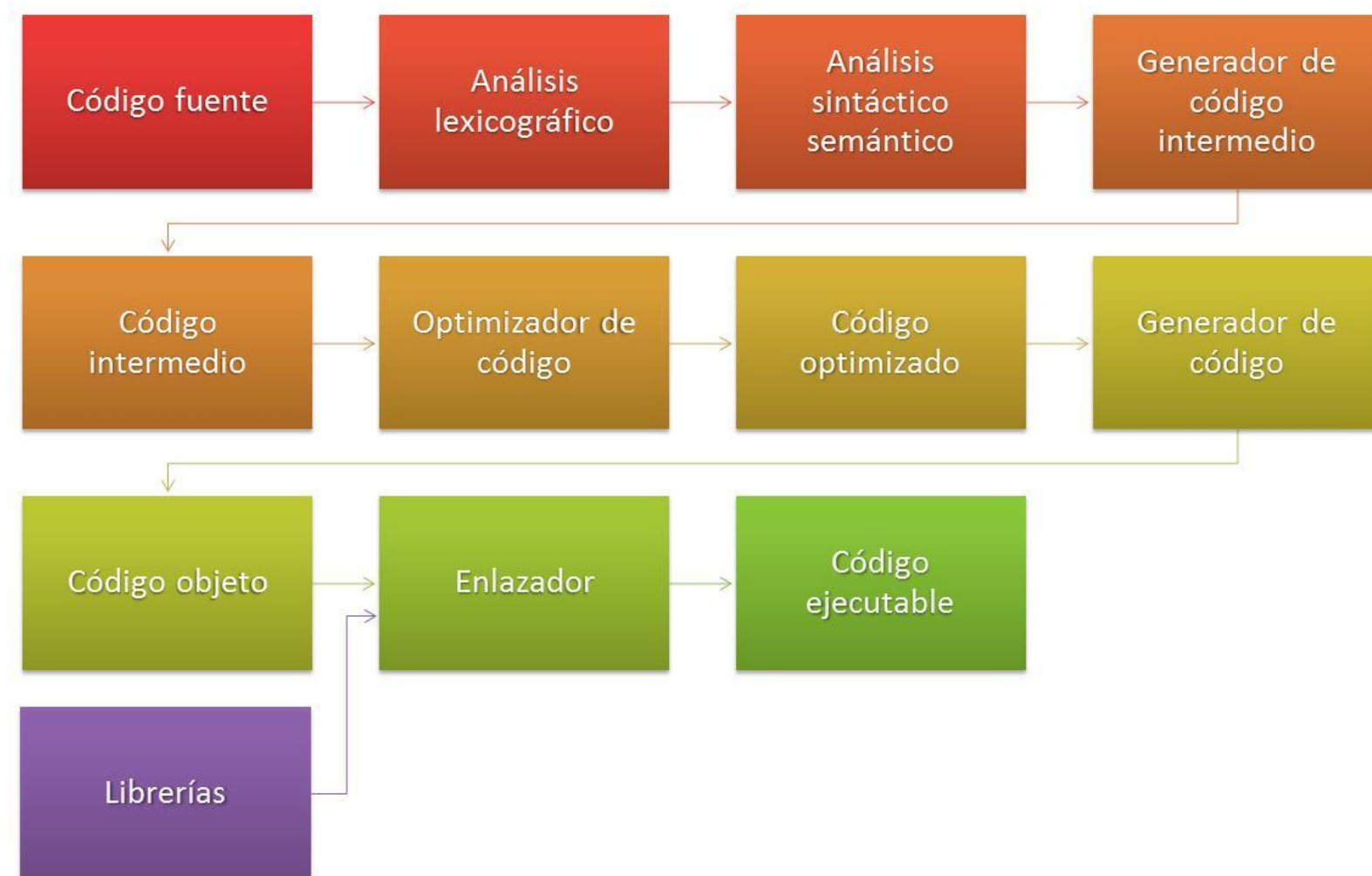
- Es genera el **codi objecte** del programa en codi màquina reubicable (amb les adreces de memòria no fixades).

6. Enllaçador de llibreries

- S'enllaça el codi objecte amb les llibreries de sistema necessàries obtenint així el **codi executable**.

1.3. Obtenció de codi executable

1.3.2. Compilació – Fases



Procés de desenvolupament de software

1.4. Processos de desenvolupament

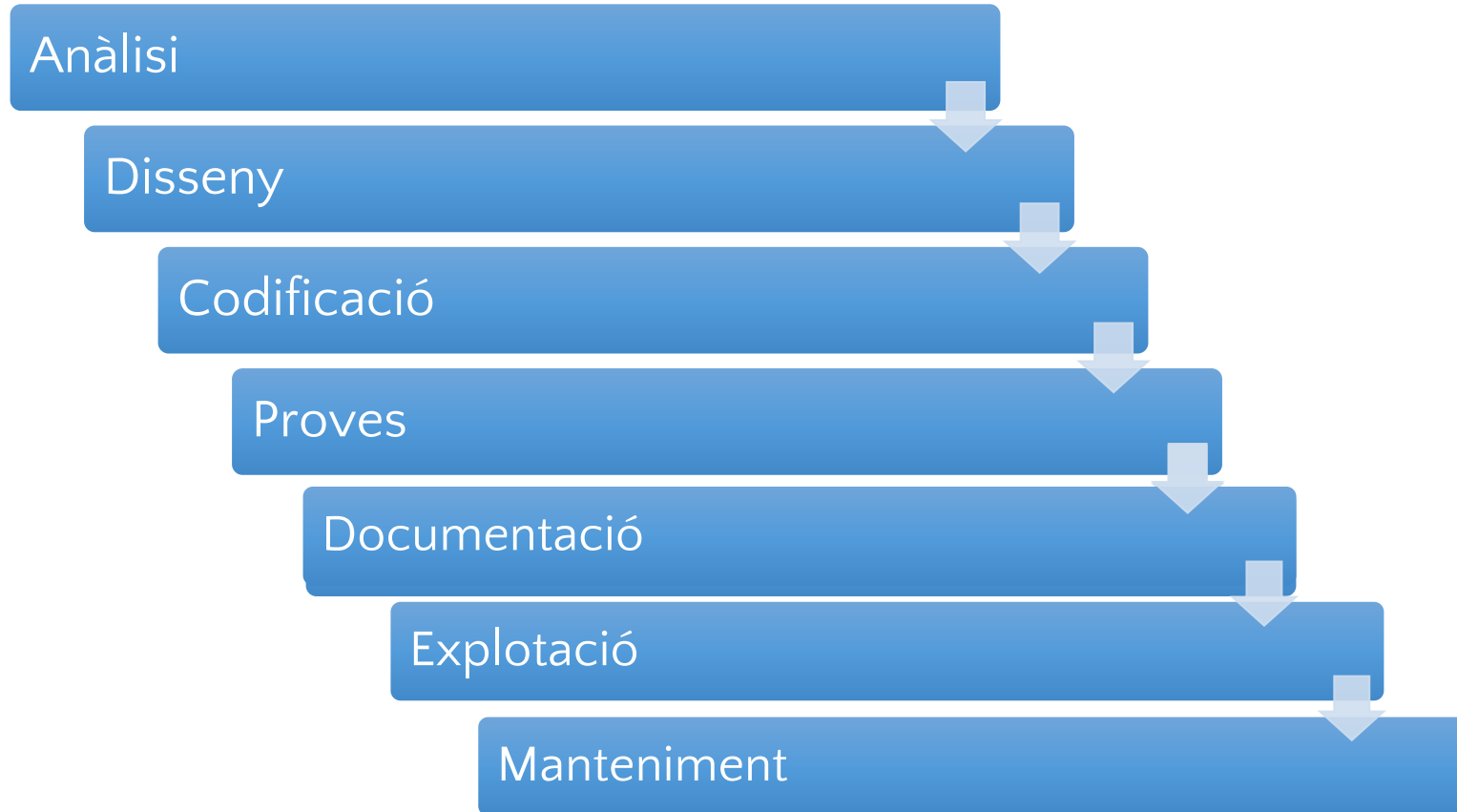
- ❑ El procés de desenvolupament de programari consta de diverses **etapes**, des que neix la necessitat del programari fins que es fa i s'utilitza.
- ❑ **Cicle de vida** del programari.
- ❑ Existeixen diferents models d'etapes de desenvolupament de programari. Un dels més estesos és el **model en cascada**.

1.4. Processos de desenvolupament

❑ Quines fases creus que existeixen en el desenvolupament del software?



1.4. Processos de desenvolupament



1.4. Processos de desenvolupament

1.4.1. Anàlisi

- ❑ Es defineixen els requeriments, per part del client, del software que haurem de desenvolupar.
- ❑ Entrevista amb el client: caldrà “investigar” les seves necessitats pel que fa al procés en qüestió.
- ❑ Moltes vegades el client no té clar o no sap expressar de manera ben definida les seves necessitats.
- ❑ Haurem de veure com fan les coses ara (abans del nou software) i què esperen del nou software.
Un bon analista haurà de saber detectar aquestes necessitats i orientar al client de com hauria de ser el seu nou programa.
- ❑ Crearem un informe ERS (Especificació de Requeriments del Sistema) junt amb un diagrama de classes o d'Entitat/Relació.

1.4. Processos de desenvolupament

1.4.1. Anàlisi

❑ Exemple d'informe de requeriments

PROPOSITO U OBJETIVOS.....	5
OBJETIVO GENERAL.....	5
OBJETIVO ESPECIFICO	5
ALCANCE.....	6
DESCRIPCION DEL PROBLEMA.....	7
HERRAMIENTAS PARA LA CAPTURA DE REQUERIMIENTOS.....	8
ANALISIS ESTADISTICO, INTERPRETACION DE LA ENCUESTA ANTERIOR DESDE LA PREGUNTA 1 A LA PREGUNTA 11.....	10
MAPA DE PROCESOS	13
5.1 PROCESO PEPSC	14
5.2 ACTIVIDADES DEL PROCESO	16
5.2.1 Lluvia de ideas	16
5.3 Documentación del proceso.....	17
DIAGRAMA DE ACTIVIDADES	18
REQUERIMIENTO DE ARQUITECTURA (DISEÑO FISICO)	19
REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES	20
ACTORES QUE INTERACTUAN CON EL SISTEMA.....	21

1.4. Processos de desenvolupament

1.4.2. Disseny

- ❑ Es determina el **funcionament global**, sense detalls.
- ❑ Es defineixen els **recursos** que necessitarà el sistema tant a nivell **físic** com **lògic** · no estem parlant només de requeriments informàtics, sinó de tots els **actors** que intervindran en el procés global: persones, comunicacions, logística, etc.
- ❑ Si hi ha restriccions econòmiques també es pot fer a l'inrevés: dissenyar el nou sistema en funció dels recursos disponibles.
- ❑ L'opció més equilibrada serà un entremig, arribant a un compromís entre aprofitar el màxim els recursos existents i la incorporació de nous recursos que aportin gran valor afegit.

1.4. Processos de desenvolupament

1.4.2. Disseny

En aquest pas definirem:

- ❑ **Diagrames** de **casos d'ús** i de **seqüència**: per definir la funcionalitat del sistema.
 - ❑ **Formats de la informació** d'E/S.
 - ❑ **Estructures de dades**.
 - ❑ **Divisió modular** del sistema.
- ❖ A partir de tots aquests documents formarem el que s'anomena **quadern de càrrega**.

1.4. Processos de desenvolupament

1.4.3. Codificació

- ❑ És la **fase més coneguda** del desenvolupament de software.
- ❑ Es tracta de **traduir** totes les necessitats, requeriments i estructures definides en els passos anteriors **a codi de programa**.
- ❑ Com més ben fet hagi estat tot l'anàlisi previ més fàcil i eficient serà el procés de passar-ho a codi.
- ❑ Per dur a terme la codificació haurem hagut de triar un **llenguatge de programació** determinat.
- ❑ L'ús d'un **entorn de programació** ens facilitarà molt la tasca de codificació mitjançant eines específiques.

1.4. Processos de desenvolupament

1.4.4. Proves

- ❑ Cal dissenyar proves per comprovar que el software fa el que ha de fer i que no es produeixen errors.
- ❑ Es fan proves en **diferents fases i tipus**, per exemple després de la documentació · aquestes les executaran **usuaris inexperts** per veure si els manuals són eficients.
- ❑ També és convenient la prova per **personal expert** però que no han participat en el desenvolupament, ja que sabran buscar ràpid els punts on pot fallar el software.
- ❑ Cal veure doncs que **el programa és robust i respon com s'espera** d'ell.

1.4. Processos de desenvolupament

1.4.5. Documentació

❑ Hi haurà dos tipus de documentació:

- Manual: adreçada a l'usuari final del programa.
- Doc. Tècnica: adreçada a l'equip de programació.

❑ Manual per a l'usuari

- La documentació d'usuari haurà de ser completa, de qualitat i “amigable”.
- Cal que mostri el funcionament de tot el programa de manera clara, guiada i intuïtiva. Qualsevol usuari novell ha de poder interpretar-la i utilitzar el programa de manera autònoma.

1.4. Processos de desenvolupament

1.4.5. Documentació

❑ Documentació tècnica

- Adreçada a l'equip de desenvolupadors.
- Explica el **funcionament intern** del programa.
- L'objectiu seria que a partir d'aquesta documentació, **qualsevol equip de programació** fos capaç d'entendre el funcionament del programa i fer-ne el manteniment.
- Hi ha projectes en que el un software **interactua amb altres aplicatius**, en aquests casos l'equip de programació dels aplicatius necessitaran la documentació tècnica del primer.

1.4. Processos de desenvolupament

1.4.6. Explotació

- ❑ És el moment de posta en marxa del programa.
- ❑ Aquesta posta en marxa la pot dur a terme l'equip de programadors (si es tracta d'un software "claus en ma") o per l'usuari final (normalment en els casos de software de distribució). En aquest segon cas caldrà documentar també com fer la instal·lació i preparar-la per tal que sigui guiada i intuitiva.
- ❑ Tinguem en compte que en els casos en que el nostre programa substitueixi un altre, haurem de valorar la possibilitat de que els dos softwares convisquin treballant en paral·lel durant un procés d'adaptació.

1.4. Processos de desenvolupament

1.4.7. Manteniment

- ❑ Tot software sol necessitar un manteniment posterior.
- ❑ Hi ha dos tipus de manteniment:
 - **Solucionar errors** del programa que es van detectant sobre la marxa a mida que es va utilitzant.
 - **Realitzar ampliacions** o modificacions del programa ja planificades a priori com a fases posteriors, o bé per cobrir noves necessitats que van sorgint amb el dia a dia.
- ❑ En aquest segon cas, tornarem a aplicar el **model en cascada**: fent un estudi de requeriments per l'ampliació, dissenyar-la, codificar, provar-la, documentar-la, implementar-la i continuar donant manteniment • veiem doncs que es tracta d'un **model cíclic** al llarg del cicle de vida del software.

Rols en el desenvolupament

1.5. Rols en el desenvolupament

- ❑ Ja hem vist que en el desenvolupament del software hi ha diverses fases, per tant, no és estrany que hi intervinguin diferents rols.
- ❑ No tenen per què ser papers rígids: una persona pot interpretar diversos papers, de vegades segons el dia, segons si és matí o tarda o sense que hi hagi una convenció preestablerta; un paper pot ser interpretat per diverses persones.

1.5. Rols en el desenvolupament – Rols habituals

❑ **Programador/a:** desenvolupa el software.

- Poden ser júnior o sènior.
- Escriuen el codi font del programa.
- Poden centrar-se en bases de dades, en part lògica o en part de presentació i components (part funcional, no de presentació, de botons, inputs de text, barres de cerca, taules, etc.).

❑ **Team leader:** rep les tasques que el seu equip ha de desenvolupar. Dona suport i dirigeix l'equip de desenvolupament que té assignat.

1.5. Rols en el desenvolupament – Rols habituals

❑ **Analista:** està en contacte amb els clients i coneix les seves necessitats i maneres de funcionar.

- Elabora documentació, manuals,
- Fa visites als clients. Junt amb el client, estableixen processos (passos que seguiria un usuari del programa i que s'haurien d'implementar per portar a terme determinades accions. Ex.: quins passos i en quins estats es troba una mostra de sang, des que s'extreu fins que s'entreguen els resultats de l'anàlisi al pacient).
- L'analista i el client arriben a consens sobre com el software a desenvolupar, pot facilitar el procés, o crear un procés diferent més eficient.

1.5. Rols en el desenvolupament – Rols habituals

❑ Analista de sistemes

- Participa en la fase d'anàlisi.
- Realitza un estudi del sistema per tal de dirigir el projecte en la direcció que garanteixi els requeriments del client.

❑ Analista programador

- Participa en les fases de disseny i codificació.
- Domina una visió més àmplia de la programació.
- Aporta una visió general del projecte a nivell de programació i es preocupa de dissenyar

1.5. Rols en el desenvolupament – Rols habituals

❑ **Arquitecte/a de software:** coneix a fons l'aplicació, o almenys el nucli central. També pot haver-n'hi més d'un i que s'especialitzin en parts concretes, si l'aplicació és molt gran.

- Participa en les fases d'anàlisi, disseny, documentació i explotació.
- És el cohesionador del procés de desenvolupament.
- Té una bona idea de possibles conseqüències que podria comportar modificar alguna part del software.
- Poden decidir com haurien de ser les taules de bases de dades o les estructures de dades principals.
- Poden no estar integrats en un equip de desenvolupament.
- Revisa tot el procés per tal que es dugui a terme de la millor manera i amb els recursos més adequats.

1.5. Rols en el desenvolupament – Rols habituals

❑ Dissenyador/a:

- D'interfícies i elements gràfics (icones, colors, tipus de lletra,...).
- D'usabilitat (de quina manera les diferents finestres procediments, serien el més intuïtiu possible i amb menys ambigüitats).

❑ Testers: poden ser a nivell d'usuari o a nivell de codi (Unit tester).

- A nivell d'usuari: proven l'aplicació com un usuari, seguint les especificacions descrites pels analistes.
- A nivell de codi: elaboren codis font que comproven de manera automàtica les funcions del programa.

1.5. Rols en el desenvolupament – Rols habituals

❑ **Cap de departament:** coordina i cuida tot el departament de desenvolupament. Es comunica amb altres sectors i nivells de l'empresa.

- Dirigeix el projecte.
- Pot ser un analista amb experiència.
- Ha de saber dirigir un equip i gestionar el temps.

Metodologies àgils

1.6. Metodologies àgils – Scrum

En els orígens, en els equips de desenvolupament, se seguia la “Metodologia en Cascada”. Amb aquesta metodologia, es planificava el desenvolupament d’una aplicació, es programava, i al final, quan ja estava fet, es presentava al client. Això, de vegades, donava lloc a sorpreses. Per exemple:

- Alguna part de l’aplicació no quedava com el client l’havia pensat.
- El client canviava d’opinió en algun dels requeriments quan aquest ja estava implementat (hores de desenvolupament perdudes, i altres parts del software poden dependre d’aquesta).
- Algun aspecte del lloc on el software s’havia d’aplicar, canviava, i el software s’havia d’adaptar a les noves circumstàncies (exemple: una regulació nova en les normes de com s’han d’etiquetar les mostres de sang, una regulació nova en els passos per declarar criptomonedes, etc.).
- Sovint es tardava més temps del planificat a implementar l’aplicació.

1.6. Metodologies àgils – Scrum

Les metodologies àgils són les més habituals avui dia, tant en software com en altres tipus de projectes. Dins d'aquestes metodologies, la Scrum és molt utilitzada. Hi ha moltes variants segons les preferències i casuístiques de cada equip.

El terme Scrum va aparèixer 8 anys abans que el terme Agile. El nom Scrum no és un acrònim (no són sigles), bé del futbol americà, per denotar que tot l'equip col·labora en un projecte comú.

1.6. Metodologies àgils – Scrum – Avantatges

L'Scrum segueix una sèrie de passos que es van iterant en intervals de temps. Aquests intervals permeten:

- Fer entregues parcials del software.
- Adaptar millor els temps que pot suposar cada funcionalitat (degut a planificacions més curtes, i per tant més precises).
- Si el client canvia d'opinió respecte a alguna funcionalitat, aquest sistema és més flexible per adaptar-se al canvi.
- Detectar aviat, desviacions de la idea original del client (detectar-se aviat, suposa menys temps corregir-ho i menys temps invertit en la desviació).

Cada iteració s'anomena **Sprint**, i freqüentment tenen una durada de 3 setmanes (2 – 5 setmanes).

1.6. Metodologies àgils – Scrum – Fases

Context d'exemple (projecte gran) (en totes les fases poden haver-hi variants)

- Equip de 6 programadors, un d'ells sènior, i un Team Leader.
- L'equip es troba en una empresa on hi ha:
 - Un altre equip de desenvolupament (igual que l'anterior).
 - Un equip de desenvolupament de components (botons, barres de cerca, etc.), també de 6 persones.
 - Un equip d'analistes de 4 persones.
 - Dos arquitectes.
 - Dos equips de testing a nivell d'usuari, cada un de 6 persones.

Un exemple d'un projecte més petit podria ser un sol equip de desenvolupament de 6, i dues persones de testing, que poden ser membres del mateix equip de desenvolupament.

1.6. Metodologies àgils – Scrum – Fases

Context d'exemple – Fases inicials

- Tenim un software per gestionar hospitals (donar d'alta pacients, personal de l'hospital, crear tipus de proves, assignar proves a pacients, guardar resultats, comunicar-se amb màquines d'anàlisi, generar informes, gestionar el personal de l'hospital, planificar visites, horaris, etc.). Aquest programa porta anys venent-se a hospitals d'arreu del món.
- Els comercials locals de la nostra empresa estan en contacte amb diferents hospitals de la zona, per saber si estan interessats en el nostre software, o si ja el tenen, en quines noves funcionalitats estarien més interessats.
- Si estan interessats, els analistes, es posen en contacte amb l'hospital, i concreten quina és la nova funció que voldrien.

1.6. Metodologies àgils – Scrum – Fases

Context d'exemple – Planificació de tasques

- 1) El/la cap de departament, 2) l'analista o analistes (després de parlar amb el client), 3) l'arquitecte/es i 4) els Team Leaders que es creu que podrien intervenir, fan una posada en comú.
- D'aquesta reunió es decideix què és el que es farà del que el client vol. Es presenta al client, que li pot donar el ok.
- Amb les funcionalitats o canvis descrits, es designen temes grans i generals. Aquesta cada un d'aquests temes es diu “Èpica”. Les èpiques es divideixen, al seu torn, en una o més “**User Stories**”. Les Èpiques i User Stories es posen al document “**Backlog**” del projecte.
- Es fa una altra reunió entre l'arquitecte/s i els Team Leaders implicats. En aquesta reunió es divideix cada User Story en tasques concretes.

1.6. Metodologies àgils – Scrum – Nivells de concreció

Les Èpiques

Fan referència a funcionalitats generals o àrees del projecte.

Exemple:

Sistema d'autenticació de l'aplicació

Carro de la compra

Sistema de notícies

Sistema de seguiment de clients

Gestió de mostres de sang

Sistema d'entrevistes a clients

Sistema de còpies automàtiques de seguretat

....

1.6. Metodologies àgils – Scrum – Nivells de concreció

Les User Stories

Es centren en funcions concretes. Són el nucli de l'Scrum.

Estructura del redactat d'una User Story:

“Com a <tipus d'usuari>, vull <algun objectiu> per tal de <algun motiu>”

Exemples:

Com a usuari client, vull fer login per entrar a l'aplicació.

Com a usuari administrador, vull una pantalla per tal de validar a usuaris client.

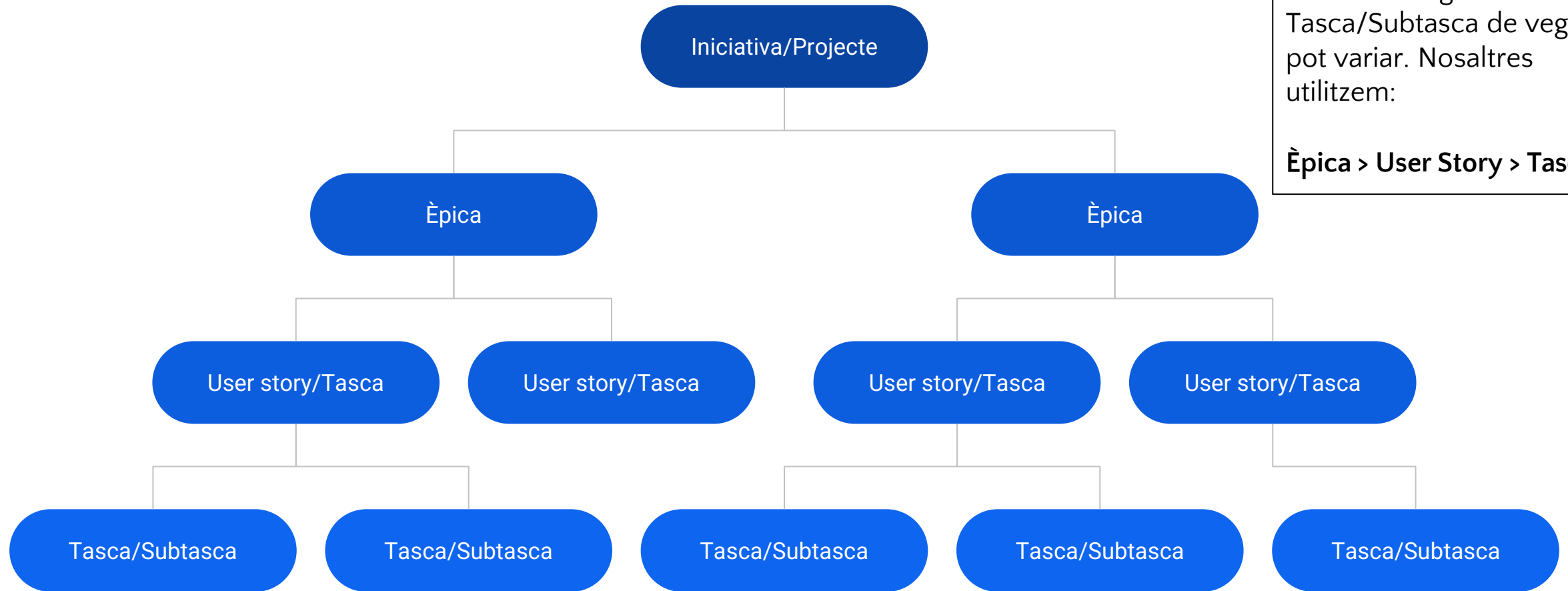
Com a comprador del producte, vull una opció del carro de la compra per afegir còpies d'un producte.

1.6. Metodologies àgils – Scrum – Nivells de concreció

Les Tasques o Subtasques

Les tasques o subtasques, són les diferents accions que caldrà fer per aconseguir completar la User Story a la qual pertanyen.

1.6. Metodologies àgils – Scrum – Nivells de concreció



La terminologia Tasca/Subtasca de vegades pot variar. Nosaltres utilitzem:

Èpica > User Story > Tasca

1.6. Metodologies àgils – Scrum – Backlog del projecte

[illegible]

1.6. Metodologies àgils – Scrum – Nivells de concreció

Activitat d'aula:

A partir de l'èpica “*Fer el carro de la compra*” d'una botiga online de venda de vins, enumerar quines User Stories hi podria haver i com es podrien desglossar en tasques.

Activitat d'aula:

En quines tasques es podria dividir la User story:

Com a usuari client, vull fer login per entrar a l'aplicació.

1.6. Metodologies àgils – Scrum – Nivells de concreció

Exemple:

User Story: *“Com a usuari client, vull fer login per entrar a l’aplicació”.*

Tasca 1: crear una pantalla de login (a nivell visual) amb un formulari amb el camp pel correu i pel password (s’inclouen imatges de com ha de ser, del tipus de lletra, etc.).

Tasca 2: validar el correu (text en format correu) i el password (entre 6 i 8 caràcters) al client. Si ok, enviar les dades al servidor. Si no ok, mostrar feedback de l’error.

Tasca 3: rebent les dades per fer login del client en el servidor, validar les dades al servidor (recuperar-les de la base de dades i comparar-les). Si ok, enviar la resposta al client de “ok”, si no ok, enviar un codi d’error al client (codi “Error”).

Tasca 4: en el client, redirigir a la següent pantalla si rep missatge “ok” del servidor. Mostrar “Error d’email o password” si rep el codi “Error” del servidor.

1.6. Metodologies àgils – Scrum – Els Sprints

Un Sprint és un període de desenvolupament. Sol durar 3 setmanes (normalment entre 2 i 5).

A l'inici de l'Sprint, el/la Team Leader, convoca una reunió de Sprint (**Scrum Meeting** o Sprint Planning Meeting). Hi són presents el Team Leader, tot el seu equip de desenvolupament, i depenent de les circumstàncies l'arquitecte/s i l'analista/es (per aclarir dubtes).

- La Scrum Meeting sol durar un matí, una tarda, o algunes vegades tot un dia. Es fa en una sala a part, tothom al voltant d'una taula.
- Es van presentant les User Stories, amb totes les Tasques desglossades. Cada Tasca s'explica per part del Team Leader, l'arquitecte o l'analista. Pot ser que, durant la reunió, arran de les converses, es creïn tasques noves (per desglossar més una User Story).

1.6. Metodologies àgils – Scrum – Els Sprints

Exemple de Scrum Meeting



1.6. Metodologies àgils – Scrum – Les votacions

Durant la reunió de Sprint, es fan **votacions sobre la durada en hores de les tasques**:

- Es reparteix un joc complet de cartes de Scrum a cada membre de l'equip de desenvolupament.
- S'anomena l'Èpica i la User Story, i es va presentant cada Tasca. Tothom pot fer preguntes i comentaris i, amb les cartes de Scrum, es fa la votació d'hores per a cada tasca.

1.6. Metodologies àgils – Scrum – Les votacions



Es poden mostrar dues cartes per sumar els números

1.6. Metodologies àgils – Scrum – Les votacions

La votació de cada Tasca:

- Després de ser presentada una Tasca, cada membre de l'equip tria una carta (sense mostrar-la als altres) per indicar quantes hores creu que suposaria aquesta tasca. Quan tothom ha escollit la carta, tothom alhora, la mostra als altres. La puntuació més alta i més baixa es descarten, i es fa una mitja de les restants. Si hi ha alguna puntuació molt distant, es pot demanar explicació (podria ser que aquell membre de l'equip hagués tingut alguna cosa en compte que la resta de l'equip no hagués considerat). El temps, resultat d'aquella Tasca, s'assigna a la Tasca.
- El/la Tema Leader també pot votar.
- Sovint, si durant les votacions una persona ha dit que una tasca es podia fer amb 4 hores i la resta han dit 8 hores o més, aquella tasca es guarda pel que ha dit 4 hores.

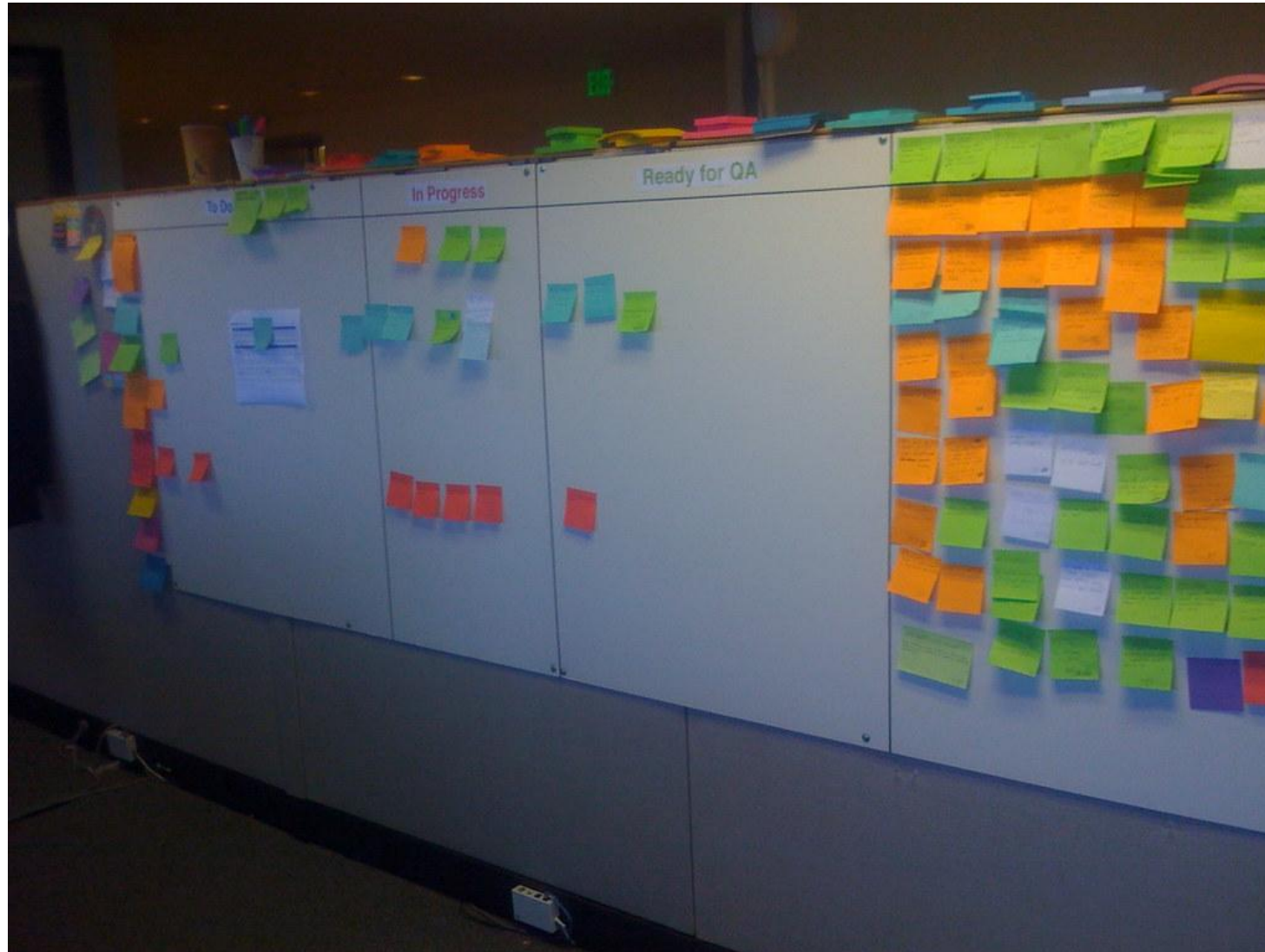
1.6. Metodologies àgils – Scrum – Les votacions

- El temps de cada tasca es va acumulant, de manera que al final de la reunió tenim el temps total que suposarien totes les tasques.
- Si un Sprint dura 3 setmanes, i hi ha 6 desenvolupadors/es a l'equip, i es fan 5 dies laborals cada una de les setmanes:
 - Són 3 setmanes x 5 dies/setmana = 15 dies
 - Si cada dia laboral té 8 hores, són 15 dies x 8 hores/dia = 120 hores
 - Si són 6 desenvolupadors/es, són 6 persones x 120 hores/persona = 720 hores
- Si el temps total que suposarien totes les tasques, és més gran que les 720 hores de l'equip, es trien les tasques que queden pel pròxim Sprint, i es fan només tasques fins que arribin a les 720 hores disponibles de l'equip.

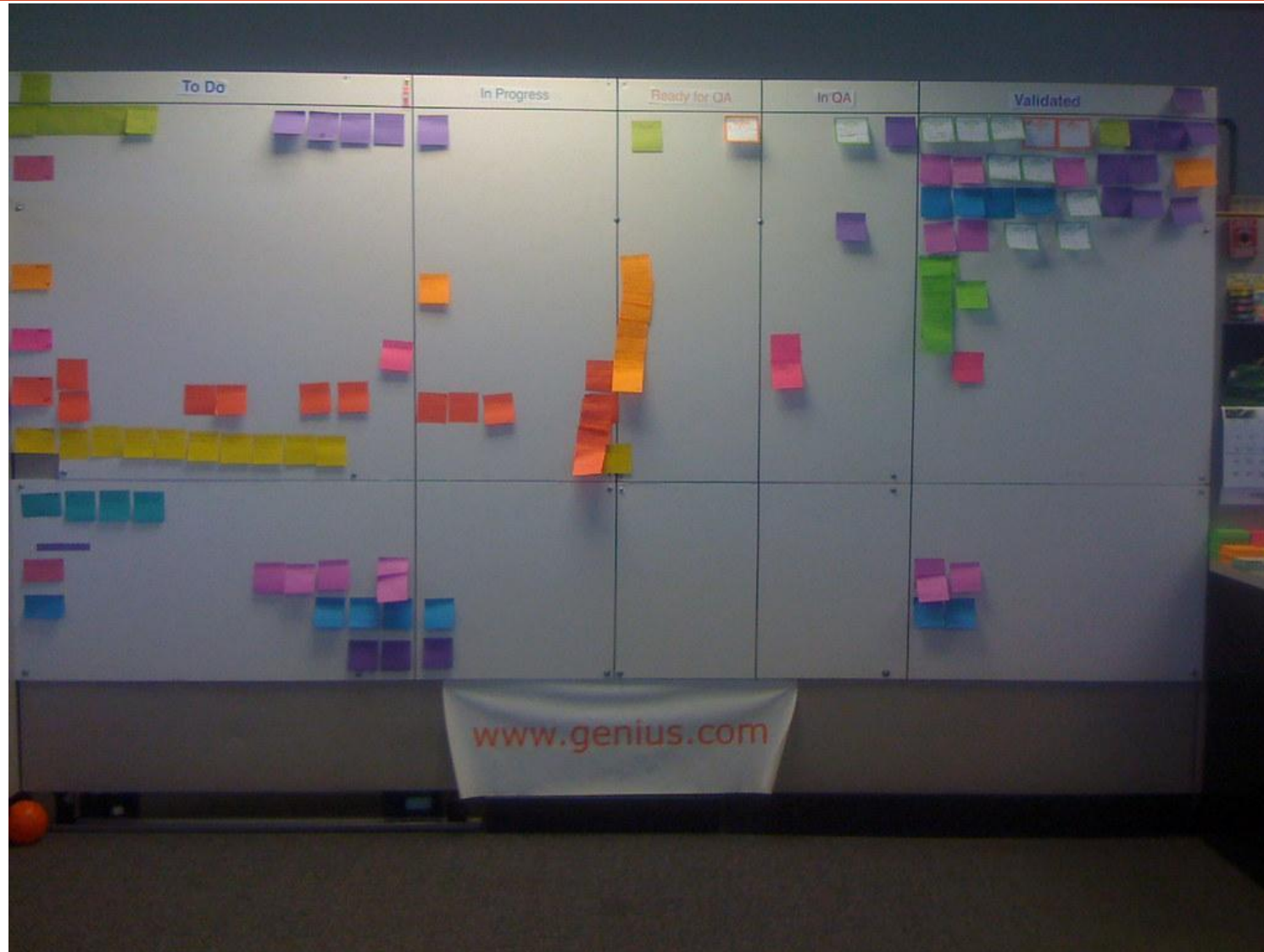
1.6. Metodologies àgils – Scrum – Tancament reunió

- Es preparen les targetes de les tasques, i la Pissarra de Scrum o Scrum Board. Les targetes poden ser post-its, o poden tenir un format digital (per exemple, Trello). Aquestes targetes/tasques es posen a la Pissarra de Scrum, en la primera columna (columna d'inici).

1.6. Metodologies à gils – Scrum – La Scrum Board



1.6. Metodologies à gils – Scrum – La Scrum Board



1.6. Metodologies àgils – Scrum – La Scrum Board

- Les columnes de la taula de Scrum poden variar entre empreses o equips.
- Exemple: “**To do**”, “**En desenvolupament**”, “**Testing intern**”, “**Testing extern**”, “**Done**”.
- Quan una **Tasca** passa de “To do”, a “En desenvolupament”, s’ha d’**assignar** a una persona (qui se’n farà càrrec). Aquesta persona pot ser qualsevol, el primer que l’agafi, però no pot agafar una tasca si ja en té una de pendent). La persona posa el seu nom a la Tasca.
- Quan una Tasca està a punt pel **testing**, l’ha d’agafar una **persona diferent** de la que ha fet el desenvolupament.
- Si una tasca no passa el testing, torna enrere, a la columna “En desenvolupament”.
- També hi ha l’opció d’utilitzar pissarres digitals (com Trello, per exemple). Tot i així, la pissarra física, dona una idea del progrés de les tasques en l’Sprint des de qualsevol lloc de la sala física.

1.6. Metodologies àgils – Scrum – Les Dailies

Cada dia al principi del matí, es fa la Daily meeting (o Daily Scrum Meeting, o Daily o Stand up Meeting).

- Hi són presents tots els membres de l'equip, a peu dret (per facilitar que duri poc temps), davant de la pissarra.
- La reunió no hauria de durar més de 5 minuts (10 a tot estirar).
- Objectiu: posar en comú de l'estat general de l'Sprint i situacions que hagin sorgit.

1.6. Metodologies àgils – Scrum – Finalitació Sprint

Es finalitza l'Sprint quan el període previst es completa. Les tasques ja estan totes a “Done” o les no acabades es passen pel següent Sprint.

Es designa unes persones de l'equip que fan la integració del que s'ha fet, al projecte. Per fer-ho, es fan servir repositoris (es veuran més endavant).

Depenent de les tasques de l'Sprint es designa alguns membres de l'equip per fer una presentació (Power Point) del que s'ha fet, a la resta de l'empresa.

Durant aquest dia, altres membres de l'equip poden fer altres tasques (variades, com arreglar errors, manteniment, etc.).

Al final de l'Sprint es fa la reunió **Sprint Retrospective**, on es reflexiona sobre què ha anat bé i què es pot millorar

1.6. Metodologies àgils – Scrum

Activitat d'aula:

Dibuixar una casa practicant Scrum.

Arquitectura de software

2. Arquitectura de software

2.1. Introducció

- L'arquitectura de software és el disseny de més alt nivell de l'estructura d'un sistema. Més enllà d'algorismes i estructures de dades.
- L'objectiu principal de l'arquitectura de programari consisteix a proporcionar elements que ajudin a la presa de decisions abstraient els conceptes del sistema mitjançant un llenguatge comú.
- Per tant, el conjunt d'eines, conceptes i elements d'abstracció s'organitzen en forma de **patrons** i **models**.

2. Arquitectura de software

2.2. Patrons de disseny

- Els patrons de disseny, també anomenats patrons de desenvolupament, estableixen els components de l'arquitectura la funcionalitat i comportament de cadascun.
- Les directrius marcades pels patrons de disseny faciliten la tasca de dissenyar un programari, encara que no la seva totalitat. Els patrons no especifiquen totes les característiques o relacions dels components al nostre programari, no que estan centrats en un àmbit específic. Cada patró determina i especifica els aspectes d'un dels tres àmbits principals:
 - Creacionals
 - Estructurals
 - De comportament

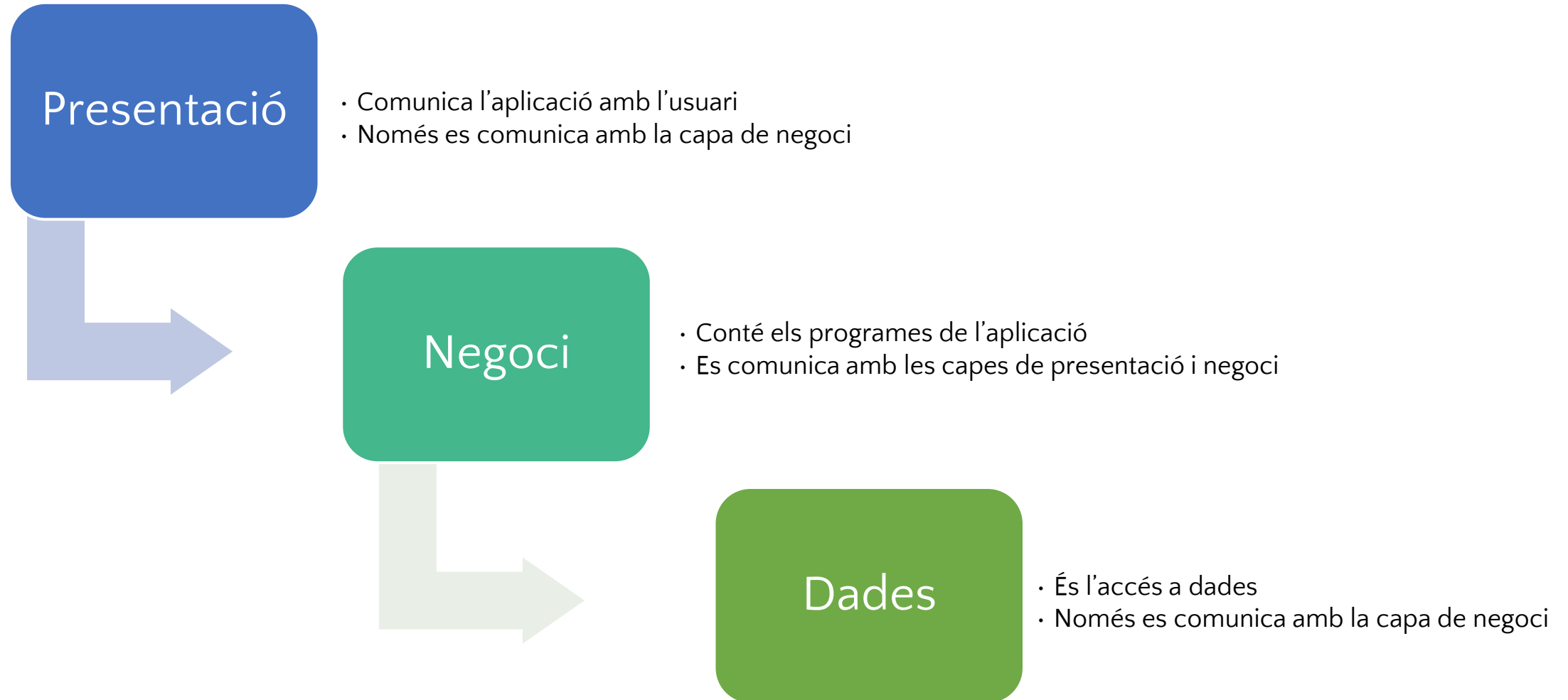
2. Arquitectura de software

2.3. Desenvolupament en tres capes

- El desenvolupament de tres capes neix de la necessitat de separar la lògica de l'aplicació del disseny, separant les dades de la presentació a l'usuari.
- Millora i facilita l'estructura del nostre *software*.
- Aporta la possibilitat d'operar amb altres sistemes aliens a la nostra aplicació.
- La principal millora del desenvolupament en tres capes radica a evitar modificar la lògica de l'aplicació per accedir a les dades.

2. Arquitectura de software

2.3. Desenvolupament en tres capes



2. Arquitectura de software

2.3. Desenvolupament en tres capes

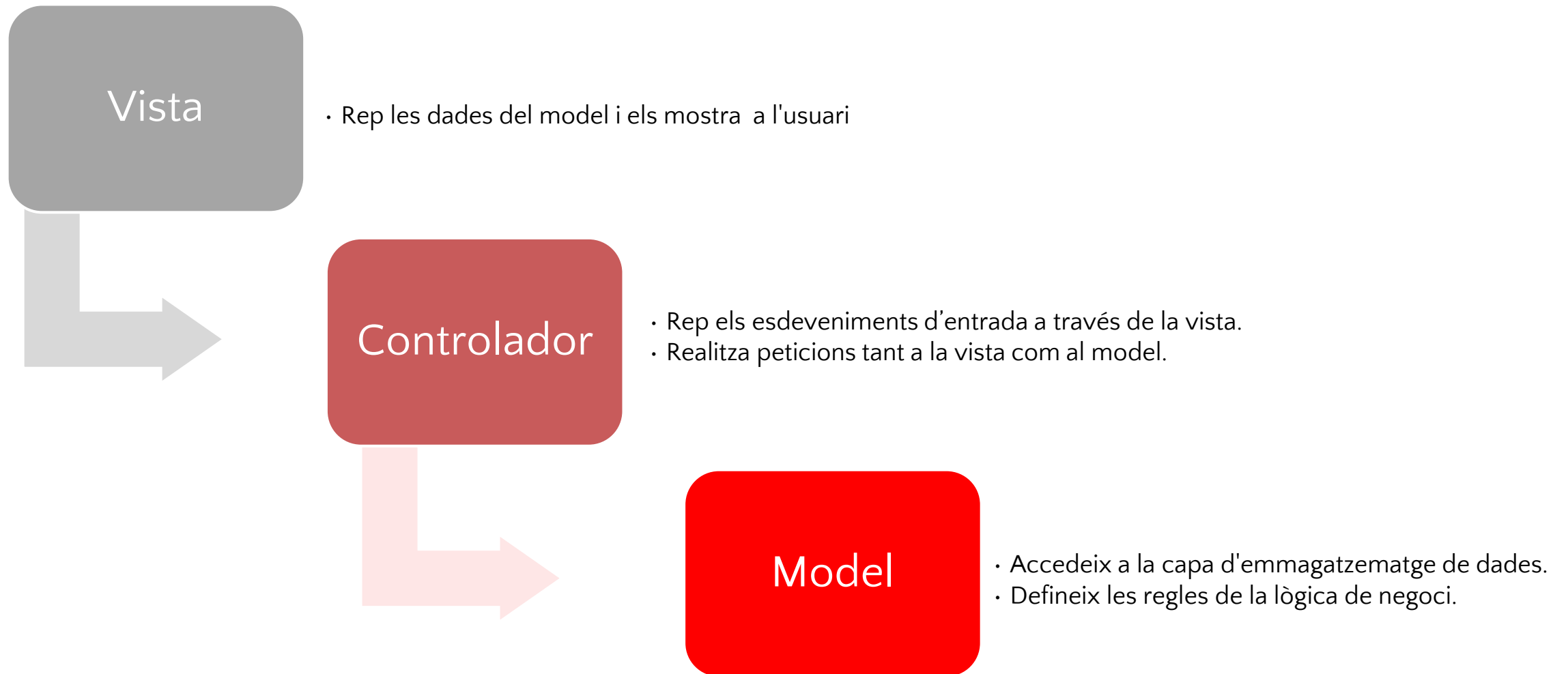
❑ Model Vista Controlador

- Dins del desenvolupament per capes, trobarem diferents models de *software*. Un es el MVC (Model vista controlador).
- Defineix tres components per les capes del desenvolupament de software.
- Organitza el codi mitjançant unes directrius utilitzant un criteri basat en la funcionalitat i no en les característiques del mateix.
- A la vista, s'estableix una connexió entre els seus components i els corresponents a la capa del model.

2. Arquitectura de software

2.4. Desenvolupament en tres capes

❑ Model Vista Controlador



2. Arquitectura de software

2.4. Desenvolupament en tres capes

- ❑ Cerca tres *frameworks* MVC per diferents llenguatges de programació.

